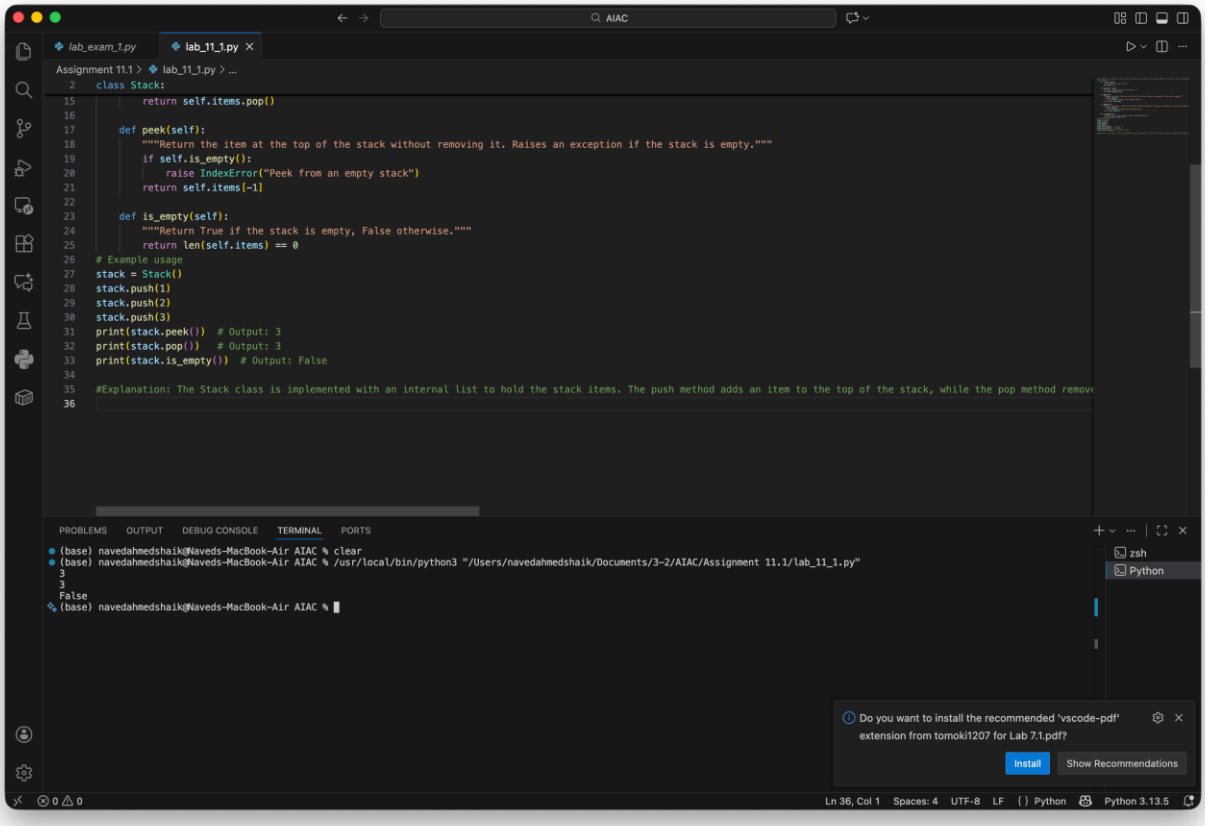
School of Computer Science and Artificial Intelligence

Lab Assignment - 11.1

Course Name : AI Assistant Coding
Name of the Student : MOHAMMAD ADIL
Registration No : 2303A54042
Batch No : 48

Task Description #1 – Stack Implementation

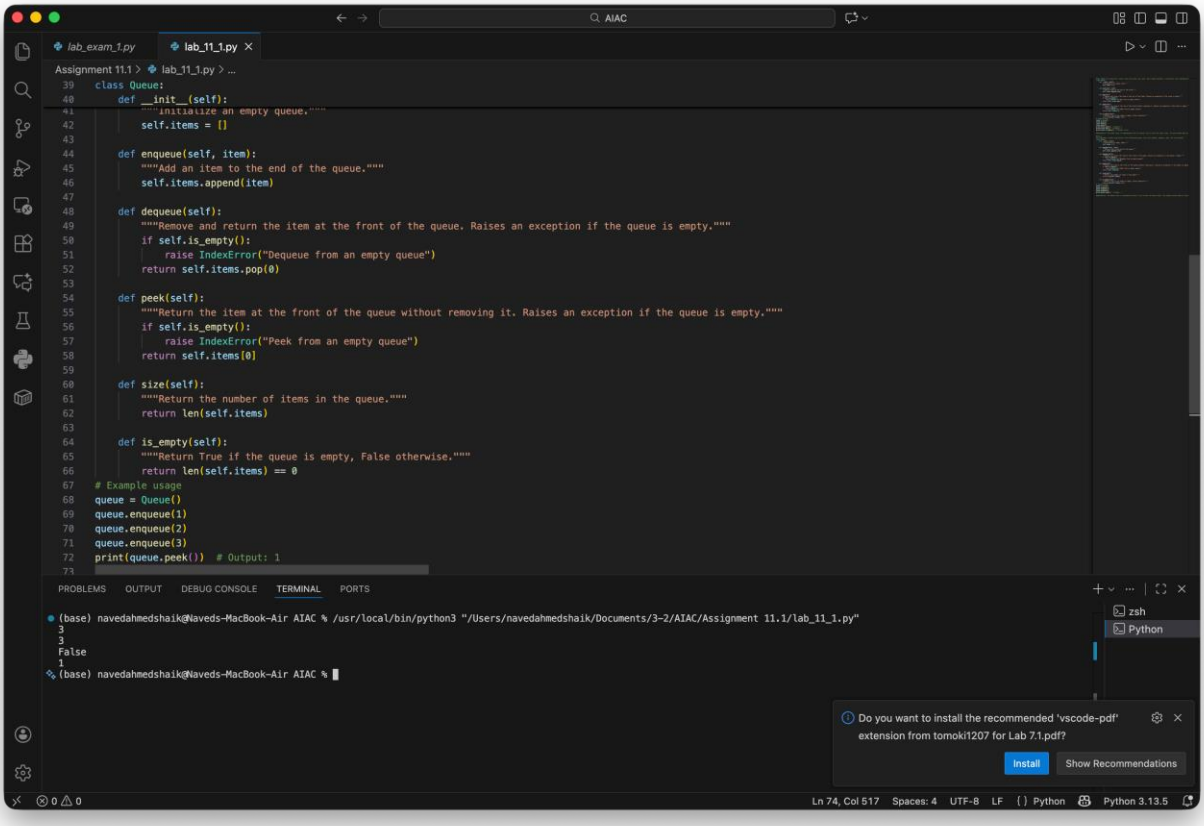
Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods. Sample Input Code: class Stack:
pass
Expected Output:
• A functional stack implementation with all required methods and docstrings.



Explanation: The Stack class is implemented with an internal list to hold the stack items. The push method adds an item to the top of the stack, while the pop method removes and returns the top item, checking for emptiness before popping. The peek method allows viewing the top item without removing it, also checking for emptiness. The is_empty method checks if the stack has any items. The example usage demonstrates how to use the Stack class.

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.
Sample Input Code:
class Queue: pass
Expected Output:
• FIFO-based queue class with enqueue, dequeue, peek, and size methods.

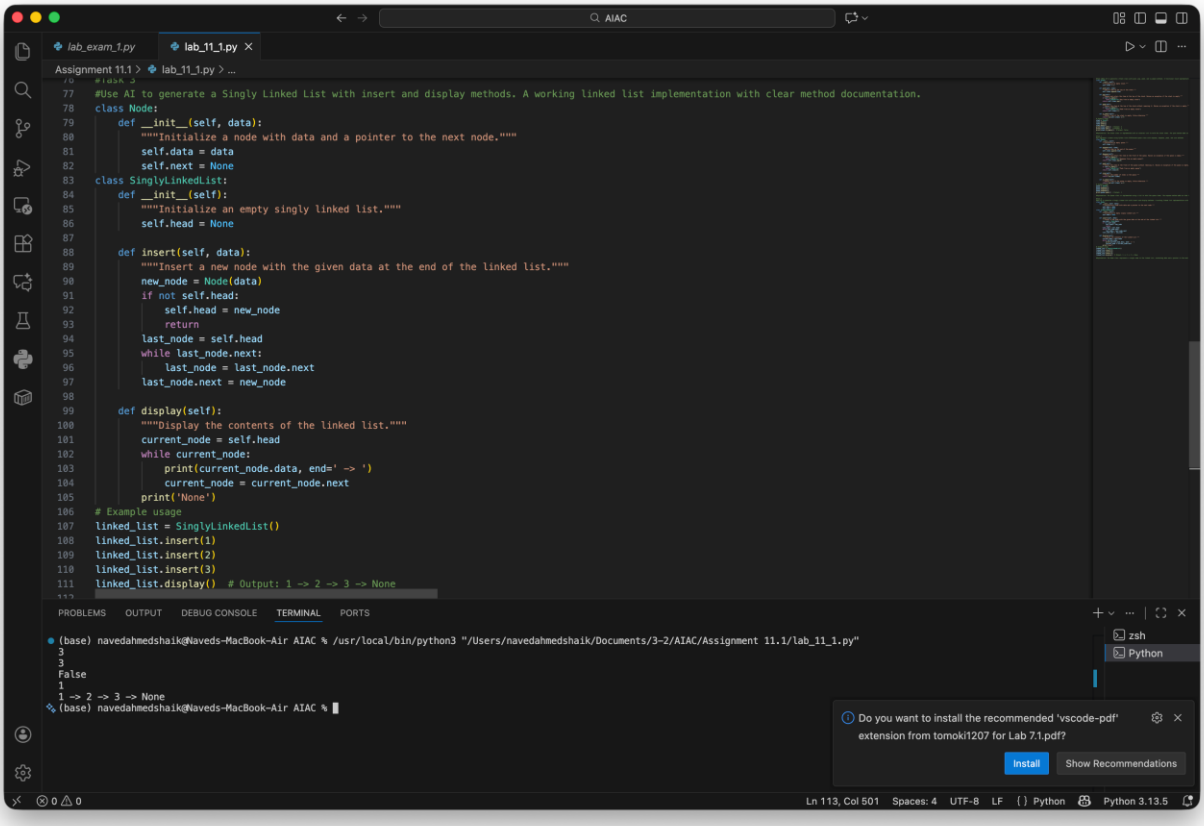


Explanation: The Queue class is implemented using a list to hold the queue items. The enqueue method adds an item to the end of the queue, while the dequeue method removes and returns the front item, checking for emptiness before dequeuing. The peek method allows viewing the front item without removing it, also checking for emptiness. The size method returns the number of items in the queue, and the is_empty method checks if the queue has any items. The example usage demonstrates how to use the Queue class.

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.
Sample Input Code: class Node: pass class LinkedList:
pass
Expected Output:
• A working linked list implementation with clear method documentation.

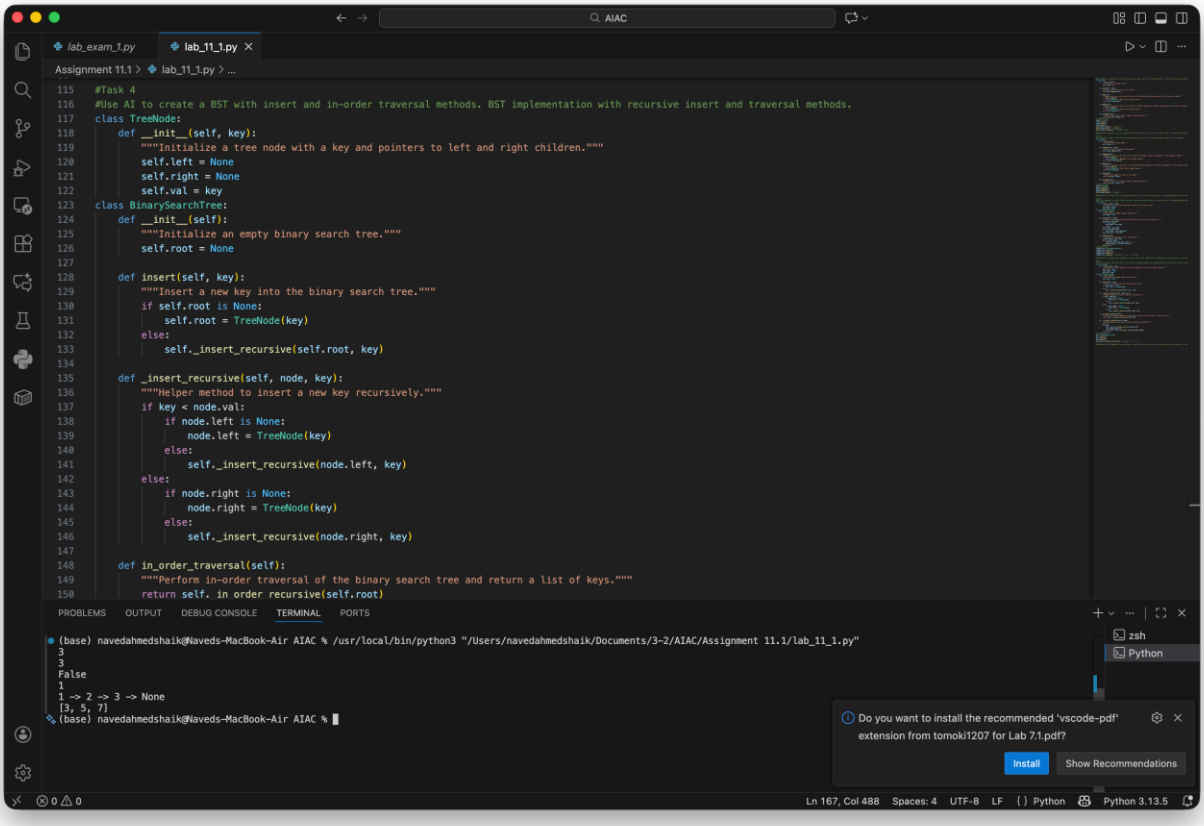
Explanation: The Node class represents a single node in the linked list, containing data and a pointer to the next node. The SinglyLinkedList class manages the linked list, allowing for insertion of new nodes at the end and displaying the contents of the list. The insert method creates a new node and adds it to the end of the list, while the display method traverses the list and prints each node's data in a readable format. The example usage demonstrates how to use the SinglyLinkedList class.



Task Description #4 – Binary Search Tree (BST)

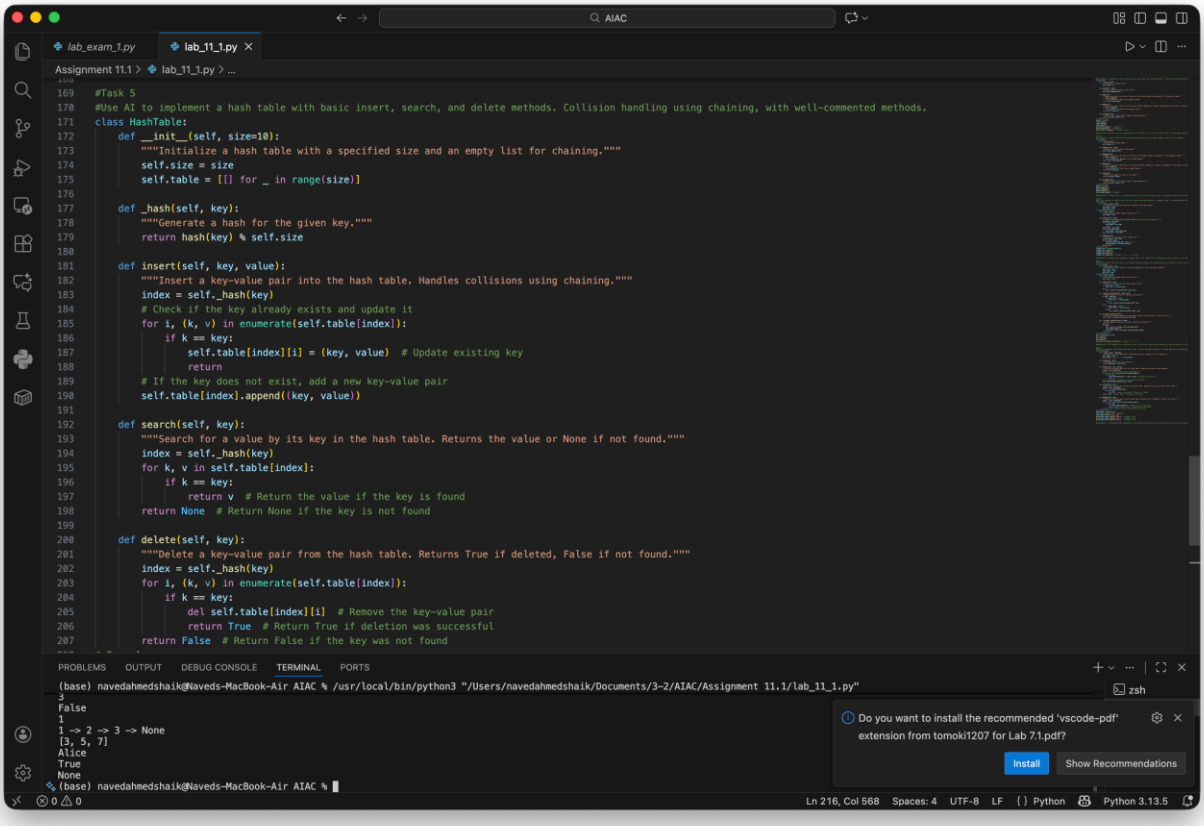
Task: Use AI to create a BST with insert and in-order traversal methods.
Sample Input Code: class BST:
pass
Expected Output:
• BST implementation with recursive insert and traversal methods.

Explanation: The TreeNode class represents a node in the binary search tree, containing a key and pointers to left and right children. The BinarySearchTree class manages the tree structure, allowing for insertion of new keys and performing in-order traversal. The insert method adds a new key to the tree, while the in_order_traversal method returns a list of keys in sorted order by traversing the tree recursively. The example usage demonstrates how to use the BinarySearchTree class.



Task Description #5 – Hash Table

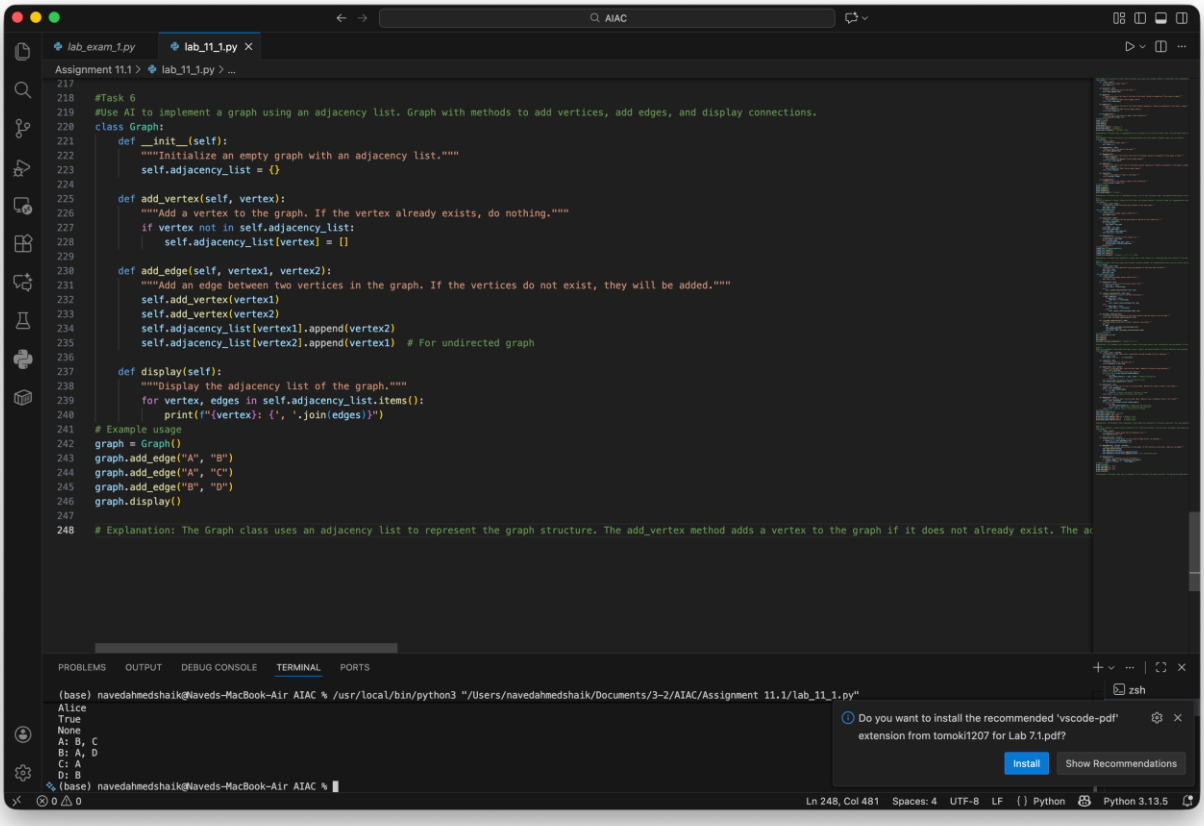
Task: Use AI to implement a hash table with basic insert, search, and delete methods.
Sample Input Code: class
HashTable:
pass
Expected Output:
• Collision handling using chaining, with well-commented methods.



Explanation: The HashTable class implements a hash table with chaining for collision resolution. The _hash method generates a hash index for a given key. The insert method adds a key-value pair to the appropriate index, updating existing keys if necessary. The search method retrieves the value associated with a key, returning None if the key is not found. The delete method removes a key-value pair from the hash table, returning True if the deletion was successful and False if the key was not found. The example usage demonstrates how to use the HashTable class.

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.
Sample Input Code:
class Graph: pass
Expected Output:
• Graph with methods to add vertices, add edges, and display connections.



Explanation: The Graph class uses an adjacency list to represent the graph structure. The add_vertex method adds a vertex to the graph if it does not already exist. The add_edge method creates an edge between two vertices, adding them to each other's adjacency list for an undirected graph. The display method prints the adjacency list, showing each vertex and its connected vertices. The example usage demonstrates how to create a graph, add edges, and display the connections.

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

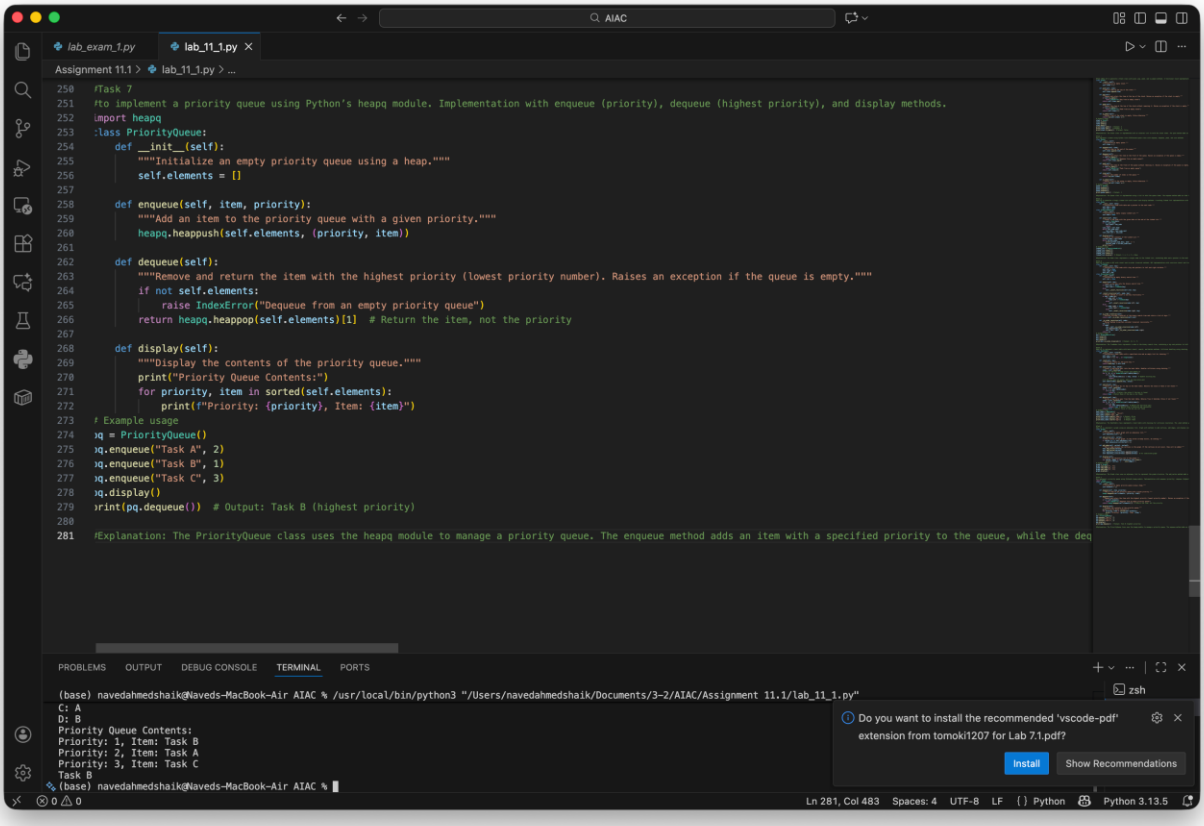
Sample Input Code: class

PriorityQueue:

pass

Expected Output:

• Implementation with enqueue (priority), dequeue (highest priority), and display methods.

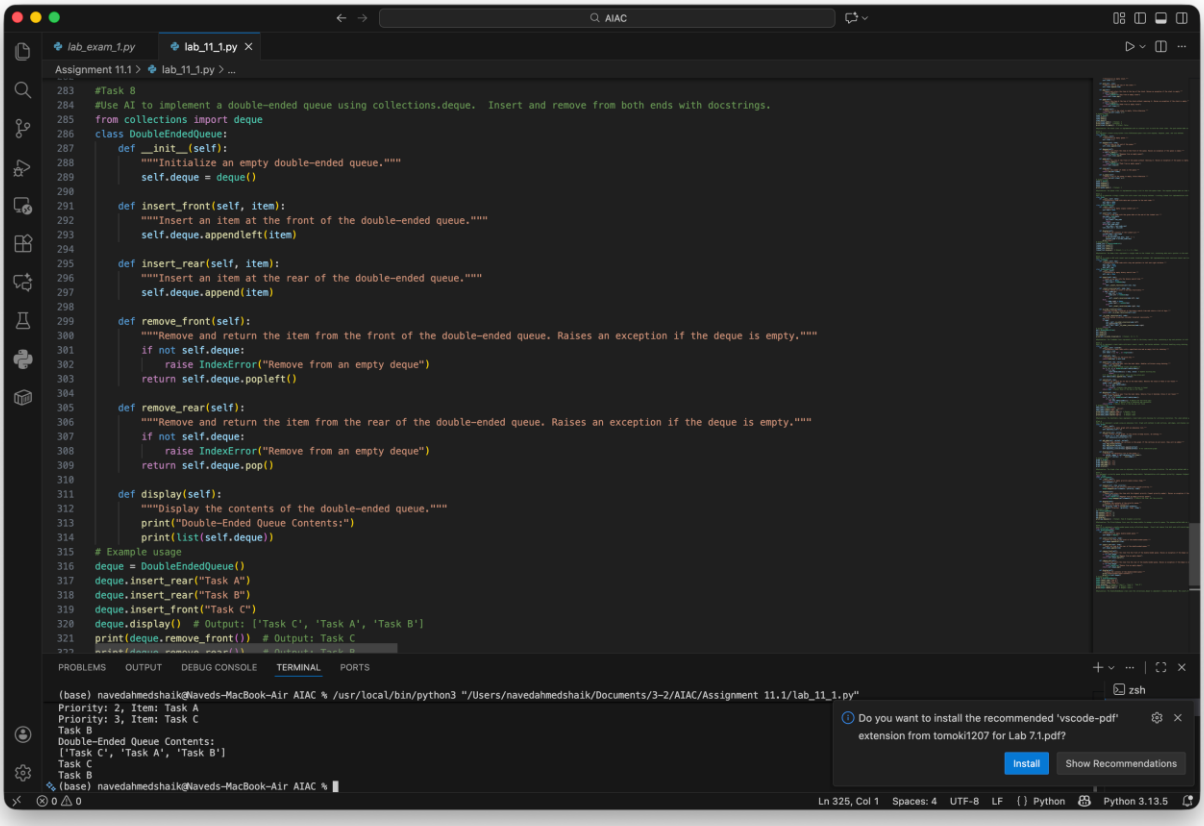


Explanation: The PriorityQueue class uses the heapq module to manage a priority queue. The enqueue method adds an item with a specified priority to the queue, while the dequeue method removes and returns the item with the highest priority (the lowest priority number). The display method shows the contents of the priority queue sorted by priority. The example usage demonstrates how to add tasks with different priorities, display the queue, and dequeue the highest priority task.

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.
Sample Input Code: class
DequeDS:
pass
Expected Output:
• Insert and remove from both ends with docstrings.

Explanation: The DoubleEndedQueue class uses the collections.deque to implement a double-ended queue. The insert_front and insert_rear methods allow adding items to the front and rear of the deque, respectively. The remove_front and remove_rear methods remove and return items from the front and rear, checking for emptiness before attempting to remove. The display method shows the current contents of the deque. The example usage demonstrates how to use the DoubleEndedQueue class to manage tasks in a double-ended queue.



Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

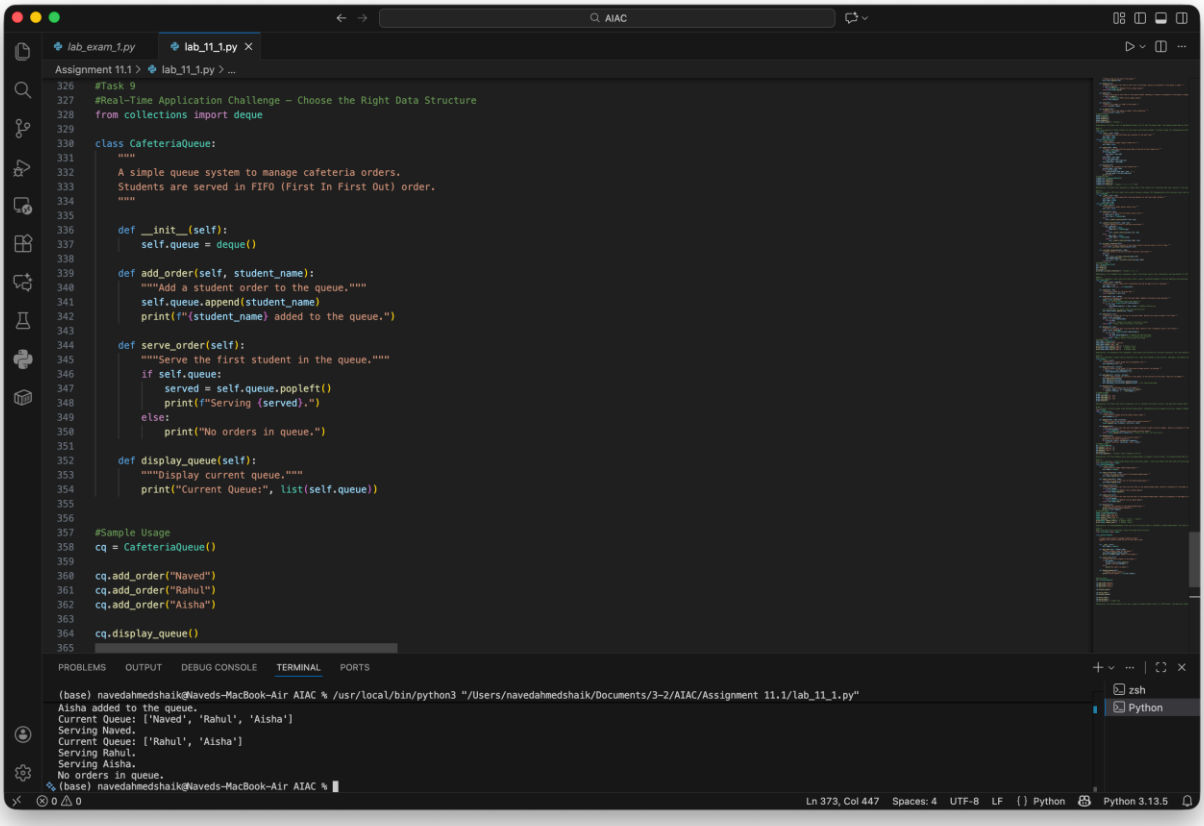
Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive. Student Task:

- For each feature, select the most appropriate data structure from the list below: o Stack o Queue o Priority Queue o Linked List o Binary Search Tree (BST) o Graph o Hash Table o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.



Explanation: The CafeteriaQueue class uses a deque to manage student orders in a FIFO manner. The add_order method adds a student's name to the end of the queue, while the serve_order method removes and serves the first student in the queue. The display_queue method shows the current state of the queue. The sample usage demonstrates how to add orders, display the queue, and serve students, including handling the case when the queue is empty.